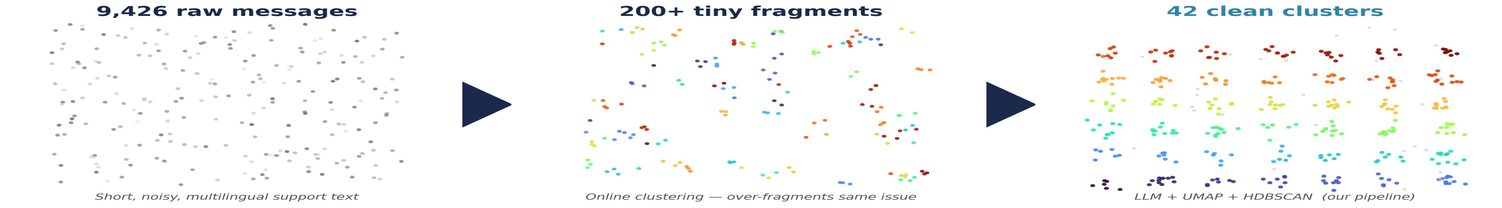
LLM-Augmented Clustering for Customer Support Ticket Triage

A Comparative Study on the ABCD Dataset

Priti Sagar · Drexel University, College of Computing & Informatics

FLAIRS-39
2026



KEY RESULT

Normalizing ticket semantics with an LLM before embedding raises coherence 0.61 → 0.78 and silhouette 0.34 → 0.52 on 500 ABCD tickets.

1. Motivation & Research Questions

Customer support platforms generate thousands of tickets daily. Grouping them into coherent issue clusters enables faster triage, root-cause analysis, and resource allocation.

What makes support text hard to cluster:

- Short, noisy, informal language
- High vocabulary variance for the same issue
- Mix of genuine issues, inquiries, and requests

Research questions:

RQ1 How do classical clustering methods compare on short-text support data?

RQ2 Does LLM-based semantic normalization before embedding improve cluster quality?

RQ3 What is the cost–quality tradeoff of integrating LLMs into the pipeline?

Real ticket samples

"hi i ordered shoes last week and got the wrong size can u help"

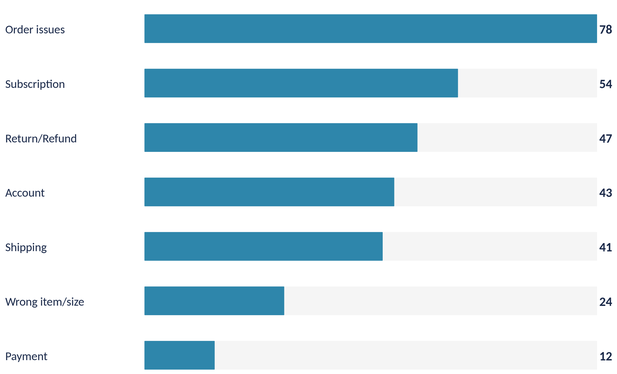
"my package says delivered but nothing is at my door?!"

2. Dataset: ABCD v1.1

A 500-ticket Zendesk-style subset of the Action-Based Conversations Dataset [Chen et al., 2021], spanning 4 languages.

Total tickets	500
Total messages	9,426
Languages	EN 400 · ES 40 · FR 40 · DE 20
Avg msgs/ticket	18.9 (min 7, max 57)
Avg words/msg	8.3 (customer: 6.3)
Date range	Aug 2025 – Feb 2026

Top issue categories (by subject keyword):



3. Four clustering methods compared

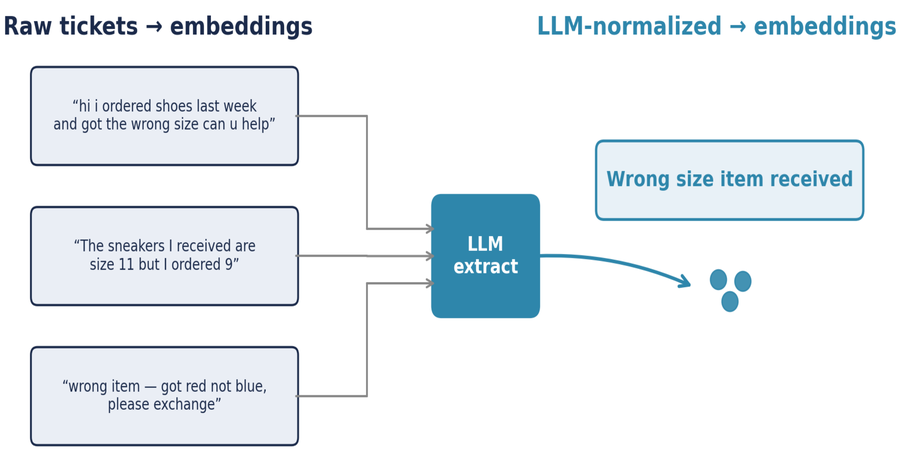
A — Online clustering
Incremental centroids · TF-IDF · over-fragments

B — K-Means / TF-IDF
Fixed k (elbow) · forces all into clusters

C — UMAP + HDBSCAN
Dense embeddings · density-based · noise-aware

D — LLM + UMAP + HDBSCAN
LLM extract → embed → UMAP → HDBSCAN
PROPOSED — BEST RESULT

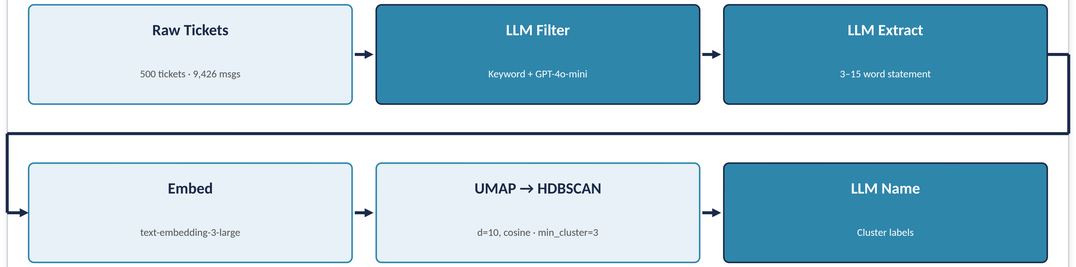
4. Key insight: normalize semantics before embedding



Three lexically different tickets describing the same issue collapse to near-identical embeddings after LLM normalization, yielding tighter clusters.

Lexically different tickets describing the same issue map to scattered points in raw embedding space. After an LLM rewrites each ticket as a short standardized statement (e.g., "Wrong size item received"), the embeddings collapse to near-identical coordinates — yielding tighter, more coherent clusters.

5. LLM-augmented pipeline (Method D)



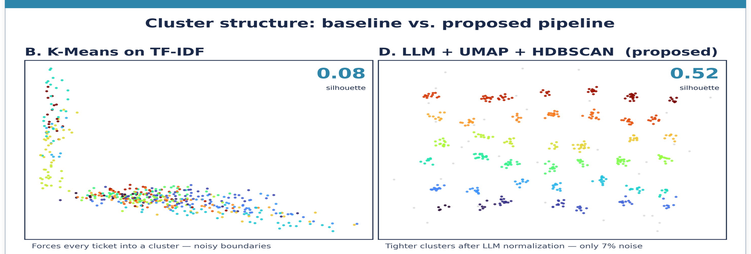
Three LLM touch-points: Filter (hybrid keyword + GPT-4o-mini, ~60% API saved) · Extract (normalized 3–15 word statement) · Name (label from 5 nearest examples)

6. Results

Metric	A	B	C	D
Silhouette ↑	—	0.08	0.34	**0.52**
# Clusters	200+	55	38	**42**
Noise %	0	0	18	**7**
Coherence ↑	0.21	0.35	0.61	**0.78**
Actionability ↑	0.15	0.28	0.55	**0.82**

Coherence & actionability rated by 3 annotators on 1–5 Likert (normalized).

7. Cluster structure: B vs. D



8. Named clusters (Method D output)

Wrong size item — 38 tickets
Representative tickets:
- *"got the shoes but wrong size"*
- *"size 11 delivered, ordered 9"*

Refund not processed — 31 tickets
Representative tickets:
- *"returned 10 days ago, no refund"*
- *"waiting for money back since last week"*

Package not received — 24 tickets
Representative tickets:
- *"says delivered, nothing at door"*
- *"tracker shows arrived, porch empty"*

Subscription still billed — 22 tickets
Representative tickets:
- *"turned off renewal, got charged"*
- *"still billed after I cancelled"*

Payment declined — 19 tickets
Representative tickets:
- *"card keeps getting declined"*
- *"payment not going through on Visa"*

Password reset failing — 17 tickets
Representative tickets:
- *"reset link never arrives"*
- *"locked out, reset not working"*

9. Conclusions & resources

0.78	0.52	~60%
coherence	silhouette	API saved
+0.17 vs C	+0.18 vs C	via hybrid filter

- LLM semantic normalization before embedding is the largest single contributor to cluster quality.
- Hybrid keyword + LLM filter saves ~60% of API calls with no recall loss.
- UMAP + HDBSCAN beats K-Means thanks to automatic cluster detection and explicit noise handling.

Source code
github.com/Priti0427/LLM-Augmented-Clustering

Single largest contributor · ~60% API saved · Automatic cluster detection

References: *[1] Chen et al., NAACL 2021. [2] McInnes et al., arXiv:1802.03426, 2018. [3] Campello et al., PAKDD 2013. [4] OpenAI, Text Embedding 3, 2024.*

Contact: pritisagar0427@gmail.com · github.com/Priti0427/LLM-Augmented-Clustering